CMPE 202 — Software Systems Engineering

Instructor: Ronald Mak | San José State University | Spring 2026

FINAL EXAM · Wednesday, May 13 2026 · 5:30 PM · BBC 130

Contents

1. Recursion

WHAT IT IS

A function solves a problem by calling *itself* on a smaller version of the same problem. Every recursive solution has exactly two parts: a
BASE CASE
(the smallest input you can solve directly, no further recursion needed) and a
RECURSIVE CASE
(break the problem into something smaller, recurse, combine the results).

WHY USE IT

Some problems are naturally self-similar — a tree is a node with subtrees, a factorial is n times factorial(n-1), a sorted array is a pivot plus two sorted halves. Writing these recursively mirrors the definition exactly. The alternative iterative version often requires an explicit stack and is harder to read.

THE DANGER

Each recursive call adds a *stack frame*. Recurse too deeply (e.g., on a 100,000-element list) and you get a stack overflow. Also, exponential recursion (like naïve Fibonacci) recomputes the same sub-problems — use memoization or iteration for those.

Pattern to memorize

```
ReturnType solve(Input x) {
    if (x is the simplest case)   // BASE CASE — no recursion
        return direct_answer;
    // RECURSIVE CASE — smaller problem + combine
    return combine(something, solve(make_smaller(x)));
}
```

Find the largest — Program15.1

```
int largest(vector<int> v) {
    if (v.size() == 1) return v[0];        // BASE: only one element left
    int first = v[0];
    v.erase(v.begin());                    // shrink the problem
    int rest  = largest(v);                // recurse on the rest
    return first > rest ? first : rest;    // combine
}
```

Tower of Hanoi — Program15.3

THE INSIGHT

To move N disks from source to destination: (1) move the top N-1 out of the way onto the temporary peg, (2) move the big disk directly to destination, (3) move the N-1 back on top. The base case is N=1 — just move it directly. Each call reduces N by 1, so it terminates. Total moves = $2^N - 1$.

```
void solve(int n, char src, char tmp, char dst) {
    if (n == 1) {                          // BASE: move one disk directly
        cout << src << " ==> " << dst << endl;
        return;
    }
    solve(n-1, src, dst, tmp);             // Step 1: move n-1 out of the way
    solve(1,   src, tmp, dst);             // Step 2: move the big disk
    solve(n-1, tmp, src, dst);             // Step 3: move n-1 back on top
}
```

Quicksort — Program15.5

DIVIDE AND CONQUER:

Pick a pivot, partition the array so everything left of the pivot is smaller and everything right is larger. The pivot is now in its final position. Recurse on the left part and the right part independently. Base case: a partition of size 0 or 1 is already sorted.

```
void Quicksort::sort(int left, int right) {
    if (right - left + 1 < 2) return;     // BASE: 0 or 1 element — done
    int pivot = partition(left, right);   // pivot is now in final position
    sort(left,      pivot - 1);           // sort left partition
    sort(pivot + 1, right);               // sort right partition
}
```

2. Backtracking

WHAT IT IS

Backtracking is recursion with an
UNDO STEP
. You explore a choice, recurse deeper, and if that path fails you *undo* the choice (backtrack) and try the next alternative. It systematically explores the entire solution space without revisiting the same state twice.

THE MENTAL MODEL

Think of a maze. You walk down a corridor. If it dead-ends, you walk back to the last junction and try the next corridor. "Walking back" is the backtrack. The key is that you restore the state exactly as it was before you made the choice.

The universal backtracking template

```
void search(State state, int depth) {
    if (is_solution(state)) {             // BASE: found a complete solution
        record(state);
        return;
    }
    for (Choice c : all_choices(state)) {
        if (is_valid(state, c)) {
            apply(state, c);              // PLACE — make the choice
            search(state, depth + 1);     // RECURSE — go deeper
            undo(state, c);               // BACKTRACK — undo the choice
        }
    }
}
```

N-Queens — Program15.7

Place N queens on an N×N board so no two queens share the same row, column, or diagonal. We fill column by column. For each column we try every row. If a row is safe, we place the queen (`occupied[row][col] = true`) and recurse to the next column. When we come back from the recursion, we *remove* the queen (`= false`) — that is the backtrack.

```
void Queens::search(int col) {
    for (int row = 0; row < SIZE; row++) {
        if (is_safe(row, col)) {
            occupied[row][col] = true;    // PLACE
            if (col == SIZE - 1)
                print();                  // All columns filled — solution!
            else
                search(col + 1);          // RECURSE to next column
            occupied[row][col] = false;   // BACKTRACK — undo
        }
    }
}
```

Sudoku — Program15.8

Walk cells left-to-right, top-to-bottom. Skip pre-filled cells. For each empty cell, try digits 1–9. If a digit is valid (not in same row, column, or 3×3 box), place it and recurse to the next cell. If the recursion returns `false`, no digit worked deeper — reset the cell to 0 and return `false` (backtrack the caller).

```
bool Sudoku::solve(int row, int col) {
    if (row == 9) return true;            // Past last row — solved!
    if (col == 9) return solve(row+1, 0); // Move to next row
    if (grid[row][col] != 0)              // Pre-filled — skip it
        return solve(row, col+1);
    for (int n = 1; n <= 9; n++) {
        if (is_ok(row, col, n)) {
            grid[row][col] = n;           // PLACE
            if (solve(row, col+1))        // RECURSE
                return true;
        }
    }
    grid[row][col] = 0;                   // BACKTRACK — no digit worked
    return false;
}
```

3. Multithreading & Race Conditions

WHAT A THREAD IS

A thread is an independent execution path inside the same process. All threads share the same heap memory and global variables, but each has its own stack. The OS scheduler decides when to pause one thread and resume another — you cannot predict or control the exact interleaving.

RACE CONDITION

When two threads read/modify a shared variable at the same time, the result depends on which thread runs first — a *race condition*. Even a simple `counter++` is actually three machine instructions (load, add, store); another thread can interrupt between any two of them and corrupt the value.

Creating and joining threads

```
#include <thread>

void print(string text) {
    for (const char& ch : text) cout << ch; // NOT SAFE without a lock
}

int main() {
    thread t1(print, "Hello, world!\n");   // Starts immediately
    thread t2(print, "Good design!\n");
    t1.join();   // Main blocks here until t1 finishes
    t2.join();   // Then blocks until t2 finishes
}   // Never let a thread go out of scope without join() or detach()
```

⚠ **Watch out:** If a `thread` object is destroyed while the thread is still running, `std::terminate()` is called — your program crashes. Always `join()` before the thread object is destroyed.

4. Mutex & lock_guard (RAII)

WHAT A MUTEX IS

A *mutex* (mutual exclusion) is a lock. Only one thread can hold it at a time. Any other thread that calls `lock()` while the mutex is held will block (sleep) until it is released. The region of code protected by the mutex is called the
CRITICAL SECTION
.

WHY LOCK_GUARD IS BETTER THAN MANUAL LOCK/UNLOCK

If you manually call `unlock()` but an exception is thrown before you reach that line, the mutex is never released — every other thread blocks forever (deadlock). `lock_guard` uses RAII: it acquires the lock in its constructor and *automatically releases it in its destructor*, which runs whenever the guard goes out of scope — normally or via exception.

BAD — Program16.2a (manual)

```
mutex print_mutex;

void print(string text) {
    print_mutex.lock();      // Manual acquire
    for (char ch : text)
        cout << ch;
    print_mutex.unlock();    // Risky: skipped on exception
}
```

GOOD — Program16.2c (RAII)

```
mutex print_mutex;

void print(string text) {
    lock_guard<mutex> guard(print_mutex);
    for (char ch : text)
        cout << ch;
}   // guard destructor releases the mutex
```

💡 **Tip:** The scope of the lock matters. Keep the critical section as short as possible — the longer you hold the lock, the longer other threads are blocked.

5. Thread Yielding

WHAT IT DOES

`this_thread::yield()` is a hint to the OS scheduler: "I'm done with the CPU for this slice — please run another thread." It does *not* release any mutex. It simply gives other threads a chance to run, improving fairness when threads are competing in a tight loop.

```
// Program16.2b — yield inside the lock
void print(string text) {
    for (int i = 0; i < COUNT; i++) {
        lock_guard<mutex> guard(print_mutex);
        for (char ch : text) cout << ch;
        this_thread::yield(); // Let another thread print next round
    }
}
```

⚠ **Watch out:** Yield is a *hint*, not a guarantee. The scheduler may ignore it and keep running the same thread. Never depend on yield for correctness — use proper synchronization for that.

6. Reader-Writer Lock (shared_mutex)

THE PROBLEM IT SOLVES

A plain `mutex` is exclusive: only one thread at a time, whether reading or writing. But reads don't conflict with each other — it's safe for 100 threads to read simultaneously. `shared_mutex` allows exactly that: many concurrent readers *or* one exclusive writer, never both at the same time.

Lock type	Who can hold it simultaneously	Use for
`shared_lock<shared_mutex>`	Multiple threads at once	Read-only operations
`unique_lock<shared_mutex>`	Exactly one thread; blocks all others	Write operations

```
// Program16.3 — ReaderWriter
shared_mutex meter_mutex;

void Meter::set_meter(int id) {          // WRITER — exclusive
    unique_lock<shared_mutex> lk(meter_mutex);
    setting = rand() % MAX + 1;
}
void Meter::log_meter(int id) {          // READER — shared, concurrent OK
    shared_lock<shared_mutex> lk(meter_mutex);
    printf("LOGGER #%d: setting=%d\n", id, setting);
}
```

7. Producer-Consumer

THE PATTERN

Producer threads generate items and put them into a shared bounded queue. Consumer threads take items out and process them. The queue is the shared resource — both sides need synchronized access. When full, producers wait. When empty, consumers wait.

```
// Program16.4 — ProducerConsumer (key class)
class SharedQueue {
public:
    bool is_empty() const { return data.size() == 0; }
    bool is_full()  const { return data.size() == capacity; }
    void enter(int v)     { data.push(v); }
    int  remove()         { int v = data.front(); data.pop(); return v; }
    mutex& get_mutex()    { return q_mutex; }
private:
    queue<int> data;
    size_t     capacity;
    mutex      q_mutex;
};

// Producer: lock → check not full → push → unlock → yield
// Consumer: lock → check not empty → pop → unlock → yield
```

💡 **Tip:** In production code, use `condition_variable` instead of yield-based polling. A condition variable puts the waiting thread to sleep and wakes it exactly when the queue state changes — much more efficient.

8. Dining Philosophers & Deadlock

THE SCENARIO

Five philosophers sit at a round table. Between each pair is one chopstick (5 total). To eat, a philosopher needs *both* the left and right chopstick. If every philosopher simultaneously picks up their left chopstick and waits for the right, no one ever eats —
DEADLOCK
.

FOUR CONDITIONS FOR DEADLOCK (ALL MUST HOLD)

1. **MUTUAL EXCLUSION**
   — only one thread can hold a resource at a time.
2. **HOLD AND WAIT**
   — a thread holds a resource while waiting for another.
3. **NO PREEMPTION**
   — resources cannot be taken away; only released voluntarily.
4. **CIRCULAR WAIT**
   — a cycle of threads each waiting on the next.

Break *any one* condition to prevent deadlock.

```
// DiningPhilosophers — key types
enum class PhilosopherState { Thinking, Hungry, Filling, Eating, Done };

class Philosopher {
    atomic<PhilosopherState> state;   // atomic: safe read/write without mutex
    mutex             mtx;
    condition_variable cv;
    bool using_chopstick() const {
        auto s = state.load();
        return s == PhilosopherState::Filling || s == PhilosopherState::Eating;
    }
};
```

💡 **Tip:** One classic solution: always acquire chopsticks in a fixed global order (e.g., lower-numbered first). This breaks circular wait.

UML Class Diagrams

WHAT UML IS

Unified Modeling Language — a standard visual notation for describing software structure and behavior. In CMPE 202 the main diagram type is the **class diagram**, which shows classes, their attributes and methods, and the relationships between them.

Step 1 — Noun / Verb / Adjective Analysis

HOW TO EXTRACT A CLASS DIAGRAM FROM A DESCRIPTION

Read the requirements or problem statement and categorize every word:

- **Nouns → candidate Classes** (things that exist in the domain)
- **Verbs → candidate Methods / Operations** (things objects do or messages they send)
- **Adjectives / state words → Attributes** (properties that describe an object)
- **Verbs between two nouns → Relationships** (associations, dependencies)

Example — parsing a requirement

```
"A Customer places an Order. Each Order contains one or more Products.
 A Product has a name, a price, and a quantity in stock."

Nouns      → Customer, Order, Product              (classes)
Verbs      → places, contains                      (relationships / methods)
Adjectives/state → name, price, quantity            (attributes of Product)

Result:
  class Customer  { void place_order(Order&); }
  class Order     { vector<Product*> items; }
  class Product   { string name; double price; int quantity; }
```

Step 2 — Class Box Anatomy

```
┌──────────────────────┐
│      ClassName       │  ← Name compartment (bold; italic if abstract)
├──────────────────────┤
│ - attribute : Type   │  ← Attributes (fields)
│ # protected : Type   │
│ + public_attr : Type │
├──────────────────────┤
│ + method(params) : Type  ← Operations (methods)
│ # hook() : void      │
│ - helper() : bool    │
└──────────────────────┘

Visibility: + public   - private   # protected   ~ package
```

Step 3 — Relationship Lines (the most-tested part)

Relationship	Line style	ASCII / Symbol	Meaning	Example
Inheritance (is-a)	Solid line + hollow triangle at parent	Child ———————▷ Parent	Child IS-A Parent	Car ——▷ Vehicle
Realization (implements)	Dashed line + hollow triangle at interface	Class - - - -▷ <<interface>>	Class implements interface	Car - - ▷ Drivable
Composition (owns, strong has-a)	Solid line + filled diamond at owner	Whole ◆——————— Part	Part cannot exist without Whole; Whole owns lifecycle	House ◆—— Room
Aggregation (has-a, weak)	Solid line + hollow diamond at container	Container ◇—— Element	Element can exist independently	Team ◇—— Player
Association (uses / knows)	Solid line, optional open arrow	A ———————→ B	A has a reference to B	Driver ——→ Car
Dependency (uses temporarily)	Dashed line + open arrow	A - - - - → B	A uses B but doesn't hold a reference	Printer - → Document

> 💡 **Tip:** Memory trick — diamond = "container". Filled ◆ = strong (part dies with whole). Hollow ◇ = weak (part survives). Triangle = "inherits from" or "implements".

Composition vs Aggregation vs Association — when to use each

Question to ask	If YES →
Does the part die when the whole is destroyed?	**Composition** ◆ (e.g. Order ◆—— LineItem)
Can the part be shared or exist independently?	**Aggregation** ◇ (e.g. Course ◇—— Student)
Does A just hold a pointer/reference to B?	**Association** → (e.g. Sport → PlayerStrategy)
Does A only use B as a parameter or local variable?	**Dependency - -→** (e.g. Client uses Factory)

Multiplicity Notation

```
  A ——— 1      exactly one
  A ——— 0..1   zero or one (optional)
  A ——— *       zero or more
  A ——— 1..*   one or more
  A ——— 2..5   between 2 and 5

Example:
  Customer 1 ——— 0..* Order    (one customer has zero or more orders)
  Order    1 ——— 1..* Product  (one order has one or more products)
```

UML for Design Patterns — what each looks like

Pattern	Key UML relationships
Template Method	Concrete ——▷ AbstractBase (inheritance); abstract methods shown in italics
Strategy	Context ——→ Strategy (association); ConcreteStrategy ——▷ Strategy (inheritance)
Observer	Subject ◇—— Observer (aggregation, 0..*); ConcreteObserver ——▷ Observer
Decorator	Decorator ——▷ Component (inheritance) AND Decorator ——→ Component (association = holds wrapped)
Composite	Composite ——▷ Component; Composite ◆—— Component (self-referential composition, 0..*)
Factory Method	Creator - - -→ Product (dependency); ConcreteCreator ——▷ Creator; ConcreteProduct ——▷ Product
Adapter	Adapter ——▷ Target (inheritance); Adapter ——→ Adaptee (association)
Singleton	Single class with static self-reference; constructor marked private (–)
State	Context ——→ State (association); ConcreteState ——▷ State; State - -→ Context (optional back-ref)

Abstract Classes and Interfaces in UML

```
Abstract class:    «abstract» or class name in italics
Interface:         «interface» stereotype above class name
                   OR a circle (lollipop) notation

// Abstract class — italicised name, abstract methods italicised

┌──────────────────┐
│  «abstract»      │
│  GameReport      │     ← or write GameReport in italics
├──────────────────┤
│ + generate()     │     ← concrete (not italic)
│ # acquire_data() │     ← abstract (italic or {abstract})
└──────────────────┘
        ▲
   ┌────┴─────┐
BaseballRpt  VolleyballRpt
```

Design Smells (Programs 1.x)

WHAT IS A DESIGN SMELL?

A symptom in code that signals a deeper design problem. Ron Mak's Programs 1.1–1.4 each demonstrate one smell and motivate why good design matters.

Smell 1.1 — Tight Coupling

```
// Driver creates Car directly — impossible to
// swap a different vehicle or mock in tests.
class Driver {
    Car my_car;    // hardwired to Car
    void start_car() {
        my_car.insert_key();
        my_car.turn_key();
    }
};
// Fix: program to an interface (OCP, Adapter DP)
```

Smell 1.2 — God Class

```
// AutomobileApp has 16 methods across
// driving, maintenance, and cleaning.
// One class → too many reasons to change (SRP).
// Fix: split into Driver, Mechanic, CarWash.
```

Smell 1.3 — Hardcoded Concrete Type

```
Cat *pet = new Cat();  // inflexible!
// Can't swap to Dog without editing this code.
// Fix: use base pointer + factory
Pet *pet = new Cat();  // or factory method
```

Smell 1.4 — Principle of Least Surprise

```
// Date(2023, 9, 2) is intuitive.
// A Date that stores string internally and
// exposes date_string() — not just toString —
// surprises the caller.
// Fix: name methods to match caller expectations.
```

THE FOUR CLASSIC SMELLS SUMMARIZED

Smell	Symptom	Which principle it breaks
Tight Coupling	A directly creates/calls B concrete type	OCP, Dependency Inversion
God Class	One class has too many unrelated methods	SRP
Hardcoded Concrete	new Concrete() embedded in client code	OCP, DI
Least Surprise	Method name doesn't match what it does	Law of Demeter, good naming

9. Single Responsibility Principle (SRP)

THE RULE

A class should have
ONLY ONE REASON TO CHANGE
. If a change to business logic A forces you to reopen and retest class X, but class X also implements logic B, then X has two responsibilities and you have a problem.

WHY IT MATTERS IN PRACTICE

When a class does too much, every change is risky — you can break unrelated functionality. Smaller, focused classes are easier to test in isolation, easier to reuse, and easier to understand.

BAD — Program2.1 (3 jobs in one)

```
class Catalogue {
    // Job 1: store books
    vector<Book*> booklist;
    void add(string title, ...);

    // Job 2: search books
    vector<Book*> find(const Book& target);

    // Job 3: string comparison utility
    static bool equal_ignore_case(
        string s1, string s2);
};
```

GOOD — Program2.5 (separated)

```
class Book      { /* data only */ };

class Attributes {
    AttributeMap map;  // metadata
};

class StringUtil {
    static bool equal_ignore_case(...);
};

class Catalogue {
    void add(Book*);
    vector<Book*> find(...); // uses StringUtil
};
```

💡 **Tip:** Ask: "If I change *X*, does this class need to change?" If two different changes would each require opening this file, the class has two responsibilities. Extract one.

10. Open-Closed Principle (OCP)

THE RULE

Software entities (classes, modules, functions) should be
OPEN FOR EXTENSION
but
CLOSED FOR MODIFICATION
. You should be able to add new behavior without editing existing, tested code.

HOW TO ACHIEVE IT

Define an abstract interface or base class. New behavior = new subclass implementing the interface. Existing code never changes. The key tool is *polymorphism* — the caller depends on the abstraction, not the concrete type.

BAD — Program4.1 (adding Motorcycle requires editing)

```
void drive(string type) {
    if (type == "car")    { /* car logic */ }
    if (type == "truck") { /* truck logic */ }
    // Adding motorcycle → edit this function!
}
```

GOOD — Program4.4 (add Motorcycle without edit

```
class MotorVehicleInterface {
public:
    virtual void start_engine() = 0;
    virtual void drive()        = 0;
    virtual void apply_brakes() = 0;
};
class Car        : public MotorVehicleInterface { ... }
class Truck      : public MotorVehicleInterface { ... }
class Motorcycle : public MotorVehicleInterface { ... }
```

11. Law of Demeter (Principle of Least Knowledge)

THE RULE

A method should only call methods on: (1) this, (2) its own fields, (3) parameters passed to it, (4) objects it created locally. In plain English:
TALK TO FRIENDS, NOT STRANGERS
. Don't reach through one object to access another.

THE "ONE DOT" HEURISTIC

If you see `a.getB().getC().doSomething()`, that's a Demeter violation — you're depending on A's internal structure (it has a B) and B's internal structure (it has a C). A change to either breaks your code.

BAD — Program5.7 (reaching into Engine)

```
// Car reaching into Engine's internals
Sparkplug* plug = engine.get_sparkplug();
plug->replace(); // Demeter violation — talking to a stranger

// Employee exposing internal pointer
Date* get_birthdate() { return birthdate; }
// Caller can now do: employee.get_birthdate()->set_year(1900);
```

GOOD — Program5.8/5.9 (delegate

```
// Car delegates to Engine — Engine ma
engine.replace_sparkplug(); // OK — on

// Employee returns a copy — internal
Date* get_birthdate() {
    return new Date(*birthdate); // co
}
```

Polymorphism & Virtual Functions (Programs 5.10, 7.x)

WHY POLYMORPHISM MATTERS FOR DESIGN PATTERNS

Every single one of the 13 GoF patterns in this course depends on polymorphism. A base pointer or reference calls the right derived-class method at runtime — this is the engine behind Template Method, Strategy, Factory, Observer, State, Visitor, Iterator, Adapter, Decorator, and Composite.

KEY KEYWORDS

Keyword	Meaning	Rule
virtual	Enables dynamic dispatch (runtime binding)	Put on base class method
= 0	Pure virtual — subclass must override	Makes the class abstract
override	Compiler checks the signature matches base	Always use in derived class
virtual ~Dtor()	Virtual destructor	Required when deleting via base pointer

Abstract base class (Mammal)

```
class Mammal {
public:
    Mammal(double w, double h)
        : weight(w), height(h) {}
    virtual ~Mammal() {}          // virtual dtor!

    virtual void eat()     = 0;   // pure virtual
    virtual void perform() = 0;   // pure virtual

    double get_weight() const { return weight; }

protected:
    void sleep() { snore(); }     // shared impl

private:
    double weight, height;
    void snore() { ... }
};
// Cannot do: Mammal m;  — abstract!
```

Concrete subclasses

```
class Human : public Mammal {
public:
    Human(double w, double h, bool ng)
        : Mammal(w, h), needs_glasses(ng) {}
    ~Human() {}

    void eat()     override { ... }  // must implement
    void perform() override { read_book(); sleep(); }

private:
    bool needs_glasses;
    void read_book() { ... }
};

class Cat : public Mammal {
public:
    void eat()     override { ... }
    void perform() override { shed(); }
private:
    double fur_factor;
    void shed() { ... }
};

// Polymorphic usage:
Mammal *m = new Human(77, 1.85, true);
m->perform();  // calls Human::perform() at runtime
delete m;      // calls Human::~Human() — safe!
```

ABSTRACT CLASS VS. INTERFACE (C++ TERMINOLOGY)

Concept	C++ form	UML notation
Abstract class	Has ≥1 pure virtual method; may have data members	Class name in *italics*
Interface	All pure virtual, no data; typically prefixed I	«interface» stereotype
Concrete class	Implements all inherited pure virtuals	Normal class box

RULE OF FIVE (WHEN YOU MANAGE RAW MEMORY)

If a class defines any one of these, it should define all five:

#	Special member	Triggered by
1	Destructor	delete ptr;
2	Copy constructor	MyClass b = a;
3	Copy assignment	b = a;
4	Move constructor	MyClass b = std::move(a);
5	Move assignment	b = std::move(a);

Use = delete to forbid, = default to compiler-generate.

Constructors, Destructors & Rule of Five (MoveSemantics, Program 7.x)

EVERY OBJECT HAS A LIFECYCLE

Creation → use → destruction. C++ gives you hooks at each end. Understanding when each special member fires is essential for managing resources safely.

Types of Constructors

```
class Date {
public:
    // 1. Default constructor (no args)
    Date() : year(0), month(0), day(0) {
        cout << "Default ctor\n";
    }

    // 2. Parameterized constructor
    Date(int y, int m, int d)
        : year(y), month(m), day(d) {  // initializer list!
        cout << "Ctor " << *this << "\n";
    }

    // 3. Copy constructor (const lvalue ref)
    Date(const Date& other)
        : year(other.year), month(other.month),
          day(other.day) {
        cout << "Copy ctor\n";
    }

    // 4. Move constructor (rvalue ref, noexcept)
    Date(Date&& other) noexcept
        : year(other.year), month(other.month),
          day(other.day) {
        other.year = other.month = other.day = 0;
        cout << "Move ctor\n";
    }

    // 5. Copy assignment operator
    Date& operator=(const Date& other) {
        if (this != &other) {
            year = other.year;
            month = other.month;
            day = other.day;
        }
        return *this;
    }

    // 6. Move assignment operator
    Date& operator=(Date&& other) noexcept {
        if (this != &other) {
            year = other.year; month = other.month;
            day = other.day;
            other.year = other.month = other.day = 0;
        }
        return *this;
    }

    // 7. Destructor
    ~Date() {
        cout << "Dtor " << *this << "\n";
    }

private:
    int year, month, day;
};
```

When each fires

```
int main() {
    Date d1;              // Default constructor
    Date d2(2026, 5, 13); // Parameterized constructor
    Date d3(d2);          // Copy constructor
    Date d4 = d2;         // Also copy constructor (init)
    d1 = d2;              // Copy assignment operator
    Date d5(std::move(d2));  // Move constructor
    d1 = std::move(d3);      // Move assignment operator
}  // d5, d4, d3, d2, d1 — Destructor (reverse order)
```

Initializer list vs body assignment

```
// Preferred: initializer list runs BEFORE body
Date(int y, int m, int d)
    : year(y), month(m), day(d) {}  // direct-init

// Less efficient: default-init then assign in body
Date(int y, int m, int d) {
    year = y; month = m; day = d;   // two steps for objects
}
// For const members or references: MUST use init list
```

= delete and = default

```
// Forbid copying entirely (Singleton):
Singleton(const Singleton&)            = delete;
Singleton& operator=(const Singleton&) = delete;

// Let compiler generate default behaviour:
Date(const Date&)            = default;
Date& operator=(const Date&) = default;
```

Virtual destructor rule

```
class Base {
public:
    virtual ~Base() {}  // MUST be virtual if subclasses exist
};

Base* ptr = new Derived();
delete ptr;  // Without virtual ~Base(), ~Derived() NEVER runs
             // → resource leak / undefined behaviour
```

RULE OF FIVE — THE COMPLETE TABLE

#	Special member	Signature	When triggered
1	Destructor	~T()	Scope ends, delete
2	Copy constructor	T(const T&)	T b = a; or pass by value
3	Copy assignment	T& operator=(const T&)	b = a; (both already constructed)
4	Move constructor	T(T&&) noexcept	T b = std::move(a); or return by value
5	Move assignment	T& operator=(T&&) noexcept	b = std::move(a);

THE RULE:

If your class owns a raw resource (heap memory, file handle, mutex) and you define *any one* of the five, define *all five* (or explicitly = delete / = default the ones you don't want). The compiler's generated defaults do shallow copies — dangerous when you have a raw pointer.

LVALUE VS RVALUE — QUICK REFERENCE

Term	Has name / address?	Example
lvalue	Yes — persists beyond expression	Date d; d;
rvalue	No — temporary, about to be destroyed	Date(2026,5,13), std::move(d)
T&	lvalue reference	binds to named object
T&&	rvalue reference	binds to temporary; enables move

MOVE SEMANTICS PURPOSE:

instead of copying a large resource (deep copy), *steal* it from the temporary — O(1) transfer instead of O(n) copy. The moved-from object is left in a valid but unspecified state.

12. Template Method Pattern

INTENT

Define the *skeleton* of an algorithm in a base class, fixing the sequence of steps. Let subclasses fill in the varying steps by overriding pure virtual methods. The overall flow is locked; only the details vary.

WHEN TO USE IT

When multiple classes do the same sequence of operations but differ in some steps — e.g., every report must print a header, acquire data, analyze data, print the report body, and print a footer. The header/footer are always the same; the data steps vary by report type.

BAD — Program8.1 (duplicated structure)

```
class BaseballReport {
    void generate() {
        print_header(); acquire_data();
        analyze_data(); print_report(); print_footer();
    } // Exact same sequence as VolleyballReport!
};
class VolleyballReport {
    void generate() {
        print_header(); acquire_data();
        analyze_data(); print_report(); print_footer();
    } // Duplicate — changes must be made in both!
};
```

GOOD — Program8.2 (Template Method)

```
class GameReport {                      // Base
public:
    void generate_report() {            // TEMP
        print_header();                 // conc
        acquire_data();                 // abst
        analyze_data();                 // abst
        print_report();                 // abst
        print_footer();                 // conc
    }
protected:
    void print_header() const { cout << title(
    void print_footer() const { cout << "End o
    virtual void acquire_data()       = 0;
    virtual void analyze_data()       = 0;
    virtual void print_report() const = 0;
};
class BaseballReport  : public GameReport { /*
class VolleyballReport: public GameReport { /*
```

💡 **Tip: vs. Strategy:** Template Method uses *inheritance* — the algorithm skeleton is baked into the class hierarchy. Strategy uses *composition* — you inject a strategy object. Strategy can be swapped at runtime; Template Method cannot.

13. Strategy Pattern

INTENT

Define a family of algorithms, encapsulate each in its own class, and make them interchangeable. The class that uses the algorithm holds a *pointer to the strategy interface* — it doesn't know or care which concrete strategy is plugged in.

KEY BENEFIT

You can change the algorithm used by an object *at runtime* without modifying the object itself. You can also add new algorithms (new strategy classes) without touching existing code — this satisfies OCP.

```
// Program8.4 — Sports Strategy
class PlayerStrategy { public: virtual string strategy() const = 0; };
class VenueStrategy  { public: virtual string strategy() const = 0; };

// Concrete strategies
class DraftPlayers    : public PlayerStrategy { string strategy() const override { return "Draft from col
class OpenTryoutPlayers: public PlayerStrategy { string strategy() const override { return "Open tryouts"
class Stadium         : public VenueStrategy  { string strategy() const override { return "Reserve stadiu
class Gym             : public VenueStrategy  { string strategy() const override { return "Reserve gym";

class Sport {
public:
    Sport(PlayerStrategy* ps, VenueStrategy* vs) : ps(ps), vs(vs) {}

    // Runtime swap — the key feature of Strategy
    void set_player_strategy(PlayerStrategy* s) { ps = s; }
    void set_venue_strategy(VenueStrategy* s)   { vs = s; }

    string recruit() { return ps->strategy(); }
    string venue()   { return vs->strategy(); }
private:
    PlayerStrategy* ps;
    VenueStrategy*  vs;
};
```

14. Factory Method Pattern

INTENT

Define an interface for creating an object, but let *subclasses decide which concrete class to instantiate*. The creator class never names the concrete product — it just calls the factory method. This decouples the caller from the product's class name.

TWO COMMON FORMS

FORM 1 — STATIC FACTORY:

A static method on a factory class takes a parameter (e.g., an enum) and returns the right subtype. Simple, widely used. (Program7.7)

FORM 2 — VIRTUAL FACTORY METHOD:

An abstract base class defines a pure virtual `create_X()`; subclasses override it to produce their specific product. The base class calls it without knowing the concrete type. (Program9.2)

```
// Form 1 — Program7.7 ToyFactory
class ToyFactory {
public:
    static Toy* make(Kind kind) {          // Client never writes "new ToyCar"
        switch (kind) {
            case Kind::CAR:      return new ToyCar(new Roll(),   new Engine());
            case Kind::AIRPLANE: return new ModelAirplane(new Fly(), new Engine());
            case Kind::TRAIN:    return new TrainSet(new Roll(), new ChooChoo());
            default:             return nullptr;
        }
    }
};
Toy* t = ToyFactory::make(Kind::CAR);    // Caller only knows "Toy"

// Form 2 — Program9.2 AthleticsDept
class AthleticsDept {
public:
    void generate_report(const Type& type) {
        Sport* s = create_sport(type);   // Call factory method — type unknown here
        s->generate_report();
    }
private:
    virtual Sport* create_sport(const Type&) = 0;  // FACTORY METHOD
};
class VarsityDept   : public AthleticsDept {
    Sport* create_sport(const Type& t) override { return new VarsitySport(t); }
};
class IntramuralDept: public AthleticsDept {
    Sport* create_sport(const Type& t) override { return new IntramuralSport(t); }
};
```

15. Abstract Factory Pattern

INTENT

Provide an interface for creating *families of related objects* without specifying their concrete classes. Instead of one factory method, you have a factory object with *multiple* factory methods — one per product in the family. Swapping the factory swaps the entire family at once.

FACTORY METHOD VS ABSTRACT FACTORY — THE CLEAREST DISTINCTION

Factory Method: one product type, one method, subclasses override it.
Abstract Factory: a whole *product family* (e.g., PlayerStrategy + VenueStrategy together). You swap the entire factory to get a consistent family — you'd never accidentally mix VarsityPlayers with IntramuralVenue.

```
// Program9.3 — Abstract Factory
class ProvisionsFactory {                // Abstract factory interface
public:
    virtual PlayerStrategy* make_players(const Type&) = 0;
    virtual VenueStrategy*  make_venue()              = 0;
};

class VarsityFactory : public ProvisionsFactory {
public:
    PlayerStrategy* make_players(const Type& t) override { return new DraftPlayers(t); }
    VenueStrategy*  make_venue()                override { return new Stadium(); }
};

class IntramuralFactory : public ProvisionsFactory {
public:
    PlayerStrategy* make_players(const Type& t) override { return new OpenTryouts(t); }
    VenueStrategy*  make_venue()                override { return new Gym(); }
};

// Client — never mentions Varsity or Intramural concrete types
void setup(ProvisionsFactory* factory) {
    PlayerStrategy* ps = factory->make_players(Type::SOCCER);
    VenueStrategy*  vs = factory->make_venue();
    Sport sport(ps, vs);
}
setup(new VarsityFactory());    // OR new IntramuralFactory()
```

16. Adapter Pattern

INTENT

Convert the interface of one class into another interface that clients expect. Adapter lets incompatible interfaces work together. Think of it as a plug adapter — the appliance hasn't changed, the wall socket hasn't changed, the adapter in between makes them compatible.

REAL-WORLD ANALOGY

`FootballInfo` has `get_fans()` and `get_field()`. Your `AttendanceReport` only knows how to use `AttendanceData` which has `get_attendance()` and `get_venue()`. The Adapter wraps `FootballInfo` and translates its methods to match `AttendanceData`.

```
// Program10.2 — Adapter
class AttendanceReport {
private:
    // Inner adapter: wraps any data source into the expected interface
    class ConvertedData : public AttendanceData {
    public:
        ConvertedData(Venue v, int a) : venue(v), attendance(a) {}
        Venue get_venue()      const override { return venue; }
        int   get_attendance() const override { return attendance; }
    private:
        Venue venue;
        int   attendance;
    };

    void core_print(const AttendanceData& data) const; // one print method

public:
    // For FootballInfo — adapt on the fly
    void print(const FootballInfo& f) const {
        core_print(ConvertedData(f.get_field(), f.get_fans()));
    }
    // For VolleyballStats — adapt on the fly
    void print(const VolleyballStats& v) const {
        core_print(ConvertedData(v.get_gym(), v.get_crowd()));
    }
};
```

17. Facade Pattern

INTENT

Provide a single, simplified interface to a complex subsystem. The facade doesn't add new capability — it hides the complexity of coordinating multiple subsystem classes behind one clean entry point.

ANALOGY

A TV remote is a facade. You press "Watch Netflix" and it turns on the TV, switches the HDMI input, starts the streaming box, and navigates to Netflix. You don't operate each device separately.

BAD — Program10.4 (client manages complexity)

```
// Client must know 4 subsystem classes and call them in the right order
Alumni     alumni;    alumni.send_solicitations();
BoosterClubs clubs;   clubs.schedule_meetings();
Students   students;  students.collect_fees();
Admin      admin;     admin.process_grants();
```

GOOD — Program10.5 (Fac

```
class FundRaiser {
public:
    void do_fund_raising() {
        Alumni alumni;     al
        BoosterClubs clubs; c
        Students students; st
    }
};
// Client:
FundRaiser fr;
fr.do_fund_raising();
```

18. Iterator Pattern

INTENT

Provide a uniform way to traverse elements of a collection without exposing the collection's internal representation. Whether the collection is an array, a vector, a linked list, or a hash map, the client uses the same has_next() / next() interface.

WHY IT MATTERS

Without iterators, your traversal code is tied to the collection type — change from vector to map and you must rewrite the traversal. With an iterator, the traversal code stays the same; you just swap which iterator you create.

```
// Program11.2 — Iterator
class Iterator {                        // Abstract iterator interface
public:
    virtual Player* next()      = 0;
    virtual bool    has_next() const = 0;
    virtual ~Iterator() = default;
};

class VectorIterator : public Iterator {
public:
    VectorIterator(vector<Player*> ps) : players(ps), cursor(players.begin()) {}
    Player* next()      override { return *(cursor++); }
    bool    has_next() const override { return cursor != players.end(); }
private:
    vector<Player*> players;
    vector<Player*>::iterator cursor;
};

class MapIterator : public Iterator {   // Same interface, different internals
    // ... map traversal hidden inside
};

// Client — identical traversal regardless of collection type:
void print_all(Iterator* it) {
    while (it->has_next())
        cout << it->next()->name() << endl;
}
```

19. Visitor Pattern

INTENT

Let you add new operations to a class hierarchy *without modifying those classes*. Each operation becomes a new Visitor subclass. The objects "accept" the visitor and let it do its work.

DOUBLE DISPATCH — THE KEY MECHANISM

Normal method calls use single dispatch (the object's type determines which method runs). Visitor achieves double dispatch: first the node's type is used (via accept()) to call the right visit_X() on the visitor; then the visitor's type determines the actual operation. Both types participate in selecting the right method.

```
// Program11.4 — Visitor
class Node {
public:
    virtual void accept(Visitor& v) = 0;  // Step 1: dispatch on Node type
};
class Sport : public Node {
    void accept(Visitor& v) override { v.visit_Sport(this); } // calls right visitor method
};
class Game  : public Node {
    void accept(Visitor& v) override { v.visit_Game(this); }
};

class Visitor {
public:
    virtual void visit_Sport(Sport* n) = 0;  // Step 2: dispatch on Visitor type
    virtual void visit_Game(Game*   n) = 0;
};

// Add a new operation — create a new Visitor, touch nothing else:
class ScoresReportVisitor  : public Visitor {
    void visit_Sport(Sport* n) override { /* print sport scores */ }
    void visit_Game(Game*   n) override { /* print game scores */ }
};
class WinningsReportVisitor: public Visitor { /* different behavior */ };
```

💡 **Tip: When to use Visitor vs adding a method:** If the class hierarchy is *stable* (you rarely add new node types) but you frequently add new *operations*, use Visitor. If you add node types frequently, Visitor is painful (every new node type requires updating every Visitor).

20. Observer Pattern

INTENT

Define a one-to-many dependency so that when one object (the **SUBJECT**) changes state, all its dependents (the **OBSERVERS**) are notified and updated automatically. The Subject doesn't need to know anything about who its observers are or how many there are.

PUSH VS PULL

PUSH MODEL:

The Subject passes the new data as arguments to `update()`. Observers get data immediately without querying the subject. (Used in Program12.2)

PULL MODEL:

`update()` receives a reference to the subject; observers query the subject for what they need. More flexible but observers are loosely coupled to the subject's interface.

```
// Program12.2 — Observer
class Observer {
public:
    virtual void update(const string& name, int outs) = 0; // push model
    virtual ~Observer() = default;
};

class Subject {
public:
    void attach(Observer* o) { observers.push_back(o); }
    void detach(Observer* o) { /* remove from vector */ }
    void notify(const string& name, int outs) const {
        for (Observer* o : observers)
            o->update(name, outs);          // Notify all registered observers
    }
private:
    vector<Observer*> observers;
};

// Adding a new observer type requires zero changes to Subject:
class GraphReport   : public Observer { void update(...) override { /* draw graph */ } };
class LogReport     : public Observer { void update(...) override { /* write log  */ } };
class FanClubReport : public Observer { void update(...) override { /* post tweet */ } };
```

21. State Pattern

INTENT

Allow an object to change its behavior when its internal state changes. Instead of one class with a big `switch(state)` in every method, each state becomes its own class. The object delegates all behavior to its current state object.

WHY SWITCH STATEMENTS ARE BAD HERE

Every new state requires editing every method that has a switch. Miss one and you have a bug. With the State pattern, each state class is self-contained — adding a new state means adding one new class, touching nothing else.

BAD — Program13.1 (switch everywhere)

```
enum State { READY, VALIDATING, SOLD, SOLD_OUT };

void insert_card() {
    switch (state) {
        case READY:      state = VALIDATING; break;
        case VALIDATING: cout << "Card already in"; break;
        case SOLD_OUT:   cout << "Machine empty";   break;
    }
}
void take_ticket() {
    switch (state) {     // Another switch — must maintain both!
        ...
    }
}
```

GOOD — Program13.2 (State objec

```
class State {
public:
    virtual State* insert_card()   =
    virtual State* check_validity()=
    virtual State* take_ticket()   =
};

class READY : public State {
    State* insert_card() override { r
    State* take_ticket() override {
        cout << "Insert card first";
    }
};

// Context — delegates everything to
class TicketMachine {
    State* current = new READY(...);
    void insert_card() { current = cu
    void take_ticket() { current = cu
};
```

22. Singleton Pattern

INTENT

Ensure that a class has *exactly one instance* and provide a global access point to it. Common uses: logging, configuration, connection pools, thread pools.

HOW IT WORKS

The constructor is *private* — nobody can write `new Singleton()`. A static method `get_instance()` creates the one instance on first call (lazy initialization) and returns it on every subsequent call. Copy constructor and copy assignment are deleted to prevent cloning.

```
// Standard modern C++ Singleton (thread-safe since C++11)
class Logger {
public:
    static Logger& get_instance() {
        static Logger instance;  // Created once; destroyed at program exit
        return instance;
    }
    void write(const string& msg) { cout << "[LOG] " << msg << endl; }

    Logger(const Logger&)            = delete; // No copy
    Logger& operator=(const Logger&) = delete; // No assignment
private:
    Logger() {}   // Private constructor — no external new
};

// Usage — always the same instance:
Logger::get_instance().write("App started");
Logger& log = Logger::get_instance();
log.write("Step 2");
```

23. Composite Pattern

INTENT

Compose objects into tree structures to represent part-whole hierarchies. Clients treat individual objects (leaves) and groups of objects (composites) *uniformly* through a common interface — they don't need to know which one they have.

THE KEY INSIGHT

A `ProvisionGroup` implements the same interface as a `ProvisionItem`. So a group can contain other groups — the tree can be arbitrarily deep. The recursive `get_cost()` call automatically sums the entire tree without the caller knowing its structure.

```
// Program14.4 — Composite
class ProvisionItem {                       // Component — leaf or composite
public:
    virtual double get_cost() const = 0;
    virtual void   add(ProvisionItem* item) {
        throw logic_error("Leaf cannot have children");
    }
    virtual ~ProvisionItem() = default;
};

class Meal : public ProvisionItem {         // LEAF — no children
public:
    Meal(string n, double c) : name(n), cost(c) {}
    double get_cost() const override { return cost; }
private:
    string name; double cost;
};

class ProvisionGroup : public ProvisionItem {  // COMPOSITE — has children
public:
    void add(ProvisionItem* item) override { children.push_back(item); }
    double get_cost() const override {         // Recursively sums the tree
        double total = 0;
        for (auto* c : children) total += c->get_cost(); // same call on leaf or group
        return total;
    }
private:
    vector<ProvisionItem*> children;
};

// Build a tree:
ProvisionGroup* team = new ProvisionGroup();
team->add(new Meal("Breakfast", 12.00));
team->add(new Meal("Lunch",     18.50));

ProvisionGroup* event = new ProvisionGroup();
event->add(team);                           // Group inside group
event->add(new Meal("Awards Dinner", 35.00));

cout << event->get_cost(); // 65.50 — no knowledge of tree depth needed
```

24. Decorator Pattern

INTENT

Attach additional responsibilities to an object *dynamically* by wrapping it. Decorators provide a flexible alternative to subclassing — instead of N×M subclasses for N base types and M enhancements, you write N base classes and M decorators that can be combined in any order.

STRUCTURAL KEY

A Decorator (1) *inherits from* the same base class as the object it wraps, AND (2) *holds a reference to* an object of that same base type. This is what enables chaining: each decorator wraps the previous, and all implement the same interface.

BAD — Program14.5 (boolean flags)

```
class Ticket {
    Ticket(bool party, bool vip, int coupons);
    double get_cost() const {
        double t = BASE;
        if (pregame_party) t += PARTY;
        if (vip_seating)   t += VIP;
        return t + coupons * COUPON;
    }
    // Adding new feature = edit this class
};
```

GOOD — Program14.6 (Decorator chain)

```
class Ticket { public: virtual double get_cost() const = 0; };

class Decorator : public Ticket {        // Inherits AND holds same type
public:
    Decorator(Ticket* t) : wrapped(t) {}
    double get_cost() const override {
        return my_price + wrapped->get_cost(); // delegate + add
    }
protected:
    Ticket* wrapped;
    double  my_price;
};
class Party  : public Decorator { Party(Ticket* t) : Decorator(t) { my_price=20; } };
class VIP    : public Decorator { VIP(Ticket* t)   : Decorator(t) { my_price=50; } };

// Build dynamically — any combination:
Ticket* t = new BaseTicket(80);
t = new Party(t);   // +20
t = new VIP(t);     // +50
cout << t->get_cost(); // 150
```

25. Lambda Expressions

WHAT THEY ARE

A lambda is an anonymous function defined inline at the point of use. Instead of declaring a separate named function and passing a function pointer, you write the logic directly where you need it. Lambdas can capture local variables from the surrounding scope — something plain function pointers cannot do.

```
// Syntax:  [capture](parameters) -> return_type { body }

// Lambda-1: old way - separate function needed
bool is_male(const Person& p) { return p.gender == Gender::M; }
auto males = select(people, is_male);           // function pointer

// Lambda-2: new way - inline, captures local variable
string target = "Smith";
auto smiths = select(people,
    [target](const Person& p) -> bool {        // captures 'target' by value
        return p.last == target;
    });

// Lambda-3: with STL algorithms
sort(words.begin(), words.end(),
    [](const string& a, const string& b) { return a < b; }); // alphabetical

for_each(words.begin(), words.end(),
    [](const string& s) { cout << s << endl; });
```

Capture syntax	Meaning
[]	Capture nothing — no access to local variables
[=]	Capture all local variables by value (copy)
[&]	Capture all local variables by reference
[x]	Capture only x by value
[&x]	Capture only x by reference
[x, &y]	x by value, y by reference

26. Functors (Function Objects)

WHAT THEY ARE

A functor is a class that overloads `operator()`, making its instances callable like functions. The advantage over plain functions: a functor is an *object* — it can carry state between calls, be initialized with parameters, and be stored or copied.

```
// Functor-Randomint
class RandomInt {
public:
    RandomInt(int min, int max) : min(min), max(max) { srand(time(NULL)); }
    int operator()() {                    // Makes the instance callable
        return min + rand() % (max - min + 1);
    }
private:
    int min, max;   // State persists between calls - functions can't do this
};

RandomInt roll(1, 6);           // Construct with range
cout << roll() << endl;         // Call like a function - uses operator()
cout << roll() << roll() << endl; // Each call uses the stored min/max

// Functor-Summation - accumulating state
class Summation {
public:
    void operator()(int x) { total += x; }  // Accumulates
    int get_total() const  { return total; }
private:
    int total = 0;
};

Summation sum;
for_each(v.begin(), v.end(), sum); // STL passes each element to operator()
```

27. Smart Pointers — unique_ptr

WHAT IT SOLVES

Raw pointers require manual `delete`. Forget it → memory leak. Throw an exception before reaching it → memory leak. Delete twice → undefined behavior. `unique_ptr` automates this: the object is deleted when the pointer goes out of scope, always, guaranteed.

EXCLUSIVE OWNERSHIP

Only *one* `unique_ptr` can own an object at a time. You cannot copy a `unique_ptr` (that would mean two owners both trying to delete the object). You *can*

MOVE

it — transferring ownership. After the move, the original pointer is `nullptr`.

```
// SmartUniquePointer
{
    unique_ptr<Date> p1(new Date(2001, 1, 1));  // p1 owns the Date
    // unique_ptr<Date> p2 = p1;               // ERROR - no copy!
    unique_ptr<Date> p2 = move(p1);            // Transfer ownership
    // p1 is now nullptr; p2 owns the Date

    if (p1 == nullptr)
        cout << "p1 is null" << endl;          // Yes, it is
    cout << *p2 << endl;                        // Dereference like a raw pointer
}   // p2 goes out of scope → ~Date() called automatically - no memory leak

// Prefer make_unique (C++14) - no raw new:
auto p3 = make_unique<Date>(2024, 5, 13);

// Custom deleter:
auto deleter = [](Date* d) { cout << "Deleting\n"; delete d; };
unique_ptr<Date, decltype(deleter)> p4(new Date(2020,1,1), deleter);
```

28. Smart Pointers — shared_ptr

SHARED OWNERSHIP VIA REFERENCE COUNTING

Multiple `shared_ptr`s can point to the same object. Each copy increments a reference count. Each destruction decrements it. When the count reaches 0, the object is deleted automatically. You never have to worry about who deletes it.

```
// SmartSharedPointer
shared_ptr<Date> p1(new Date(2001, 1, 1));  // ref count = 1
shared_ptr<Date> p2(p1);                    // ref count = 2 (copy is OK!)
shared_ptr<Date> p3 = p1;                   // ref count = 3

cout << *p1 << endl;   // All three point to same Date object
cout << *p2 << endl;
cout << *p3 << endl;

p1.reset(new Date(2011, 11, 11)); // p1 now points to new Date; old Date ref count → 2
p2.reset();                       // old Date ref count → 1
p3.reset();                       // old Date ref count → 0 → deleted automatically

// Prefer make_shared:
auto p4 = make_shared<Date>(2024, 5, 13);
```

	unique_ptr	shared_ptr
Number of owners	Exactly 1	Many (ref counted)
Copyable	No — move only	Yes
Overhead	Zero (raw pointer size)	Ref count allocation
Use when	Clear single owner	Shared ownership needed

29. Move Semantics

THE PROBLEM WITH COPYING

When you assign or return a large object (a vector, a string, a buffer), C++ makes a deep copy — allocates new memory and copies all the data. If the original is about to be destroyed anyway (a temporary), that copy is pure waste.

WHAT MOVING DOES

Instead of copying the data, moving *steals* the resources (heap buffer, file handle, etc.) from the source and leaves the source in an empty but valid state. Moving a `string` with 1MB of text is O(1) — just transfer the pointer. Copying is O(n).

```
// MoveSemantics-1
string a = "Hello World";

// Copy - a keeps its data, b gets a full duplicate
string b = a;
cout << a; // still "Hello World"
cout << b; // "Hello World" (separate copy)

// Move - b steals a's buffer; a is now empty
string c = move(a);
cout << a; // "" (moved-from state - valid but empty)
cout << c; // "Hello World"

// Push_back with move - avoids copying large objects into vector
vector<string> v;
string big = "...lots of data...";
v.push_back(move(big));   // Transfer into vector - O(1) instead of O(n)
// big is now in a valid but unspecified state
```

⚠ **Watch out:** After a move, the source object is in a *valid but unspecified* state. You can still assign to it or call methods on it, but

don't rely on its value.

```
// Rule of Five: if you define any of these, define all five:
class Resource {
public:
    Resource(const Resource& other);             // copy constructor
    Resource& operator=(const Resource& other); // copy assignment
    Resource(Resource&& other) noexcept;         // move constructor
    Resource& operator=(Resource&& other) noexcept; // move assignment
    ~Resource();                                 // destructor
};
```

Quick Reference

Design Patterns at a Glance

Pattern	Category	One-line description	Lab
Template Method	Behavioral	Fixed algorithm skeleton; subclasses override steps	Program8.2
Strategy	Behavioral	Swap algorithms at runtime via composition	Program8.4
Factory Method	Creational	Subclass decides which concrete class to create	Program9.2
Abstract Factory	Creational	Creates a family of related objects as a unit	Program9.3
Singleton	Creational	Exactly one instance; private constructor + static getter	Program14.1
Adapter	Structural	Converts one interface to another	Program10.2
Facade	Structural	Simplifies a complex subsystem behind one interface	Program10.5
Composite	Structural	Tree of leaf/group objects — uniform interface	Program14.4
Decorator	Structural	Wraps an object to add behavior; inherits + holds same type	Program14.6
Iterator	Behavioral	Uniform traversal of any collection via has_next/next	Program11.2
Visitor	Behavioral	Add operations to hierarchy without modifying classes	Program11.4
Observer	Behavioral	Subject notifies all registered observers on state change	Program12.2
State	Behavioral	Object delegates behavior to current state object	Program13.2

OOP Principles

Principle	Violation sign	Fix
Single Responsibility	Class has >1 reason to change	Extract class per responsibility
Open-Closed	Adding a feature requires editing existing class	Depend on abstraction; add new subclass
Law of Demeter	`a.getB().getC().doThing()`	Delegate: `a.doThing()`

Threading Quick Reference

Scenario	Tool	Notes
Protect a critical section	`mutex` + `lock_guard`	RAII; always use lock_guard
Multiple concurrent readers	`shared_mutex` + `shared_lock`	Readers do not block each other
Exclusive write	`shared_mutex` + `unique_lock`	Blocks all readers and writers
Thread-safe single value	`atomic<T>`	No mutex needed for simple types
Wait for condition	`condition_variable`	Sleep until notified; no busy-wait
Voluntary CPU release	`this_thread::yield()`	Hint only — scheduler may ignore

Recursion/Backtracking Checklist

Recursion

- Identify the base case first
- Make the problem strictly smaller each call
- Combine results on the way back up
- Watch stack depth for large inputs

Backtracking

- Place → Recurse → **Undo**
- Undo must restore state exactly
- Return false / break on failure
- Check constraints before placing

Smart Pointers

- unique_ptr: one owner, move-only
- shared_ptr: many owners, ref count
- Prefer make_unique / make_shared
- After move(), source is null/empty